

Project Development Document

Phase 5: Technical Setup, Folder Structures & Coding Guidelines

PROJECT PLATFORM: MediConnect - Premium MERN Doctor Appointment Platform

REFERENCE PATH: VIP-C2-BOOK-A-DOCTOR

DATE OF EXECUTION: June 16, 2026

AUTHOR / CREATOR: Antigravity AI Engineer & pairing USER

DOCUMENT STATUS: **APPROVED / PRODUCTION-READY**

EXECUTIVE OVERVIEW

This phase-wise document outlines the development lifecycle, system structures, and technical specifications of MediConnect. MediConnect is a clinic-agnostic, hospital-grade scheduling ecosystem built utilizing the MERN stack (MongoDB, Express, React, Node.js). It supports secure JWT access/refresh token structures, comprehensive patient calendars, and simulated multi-modal checkout payment gates.

1. Directory Structure Mapping

MediConnect segregates frontend and backend packages to decouple dependencies and optimize build sizes. The following structural map represents the primary codebase folders:

Directory Path	Functional Purpose / Description
backend/config	Contains db.js database connection setup and seed.js data seeding scripts.
backend/middleware	Custom middleware including authentication validation and error handlers.
backend/routes	API endpoint routers mapped to respective controller methods.
frontend/src/pages	Core views: Home, SearchDoctors, DoctorDetails, Patient/Doctor dashboards.

2. Coding Standards & Guidelines

- Controller-Route Separation: Route files must only define routing paths and middleware associations; all operational request handling must reside in controller scripts.
- Token Authentication: All client requests to private routes must inject the Bearer JWT access token in authorization headers. Token refreshes are triggered on access expiry.
- Async-Await Pattern: All database operations must utilize async-await structures with enclosed try-catch exception parsing for optimal error reporting.
- Component Decoupling: UI modules must remain stateless where possible, accepting callback props for actions to encourage reusability.

3. Development & Build Verification Flow

The workflow below represents the automated code verification lifecycle, from initial coding in local sandbox to production-ready deployments.



